

CS681 – Assignment 2

23B0920 23B1040

System Assumptions

- Closed system with fixed number of users
- Each user alternates between think time and request generation
- Requests have stochastic service time
- Think time modeled as Gaussian distribution
- Finite thread pool and bounded queue capacity
- Round-robin scheduling
- Requests may timeout or be dropped if system is overloaded

Simulation Model Overview

- Discrete Event Simulation (DES)
- Global event priority queue (min-heap based on time)
- Events drive system state transitions

Core Components:

- Users
- Requests
- Server (cores + queues)
- Event scheduler

Key Data Structures

Event Types:

- REQUEST_ARRIVAL
- REQUEST_DEPARTURE
- REQUEST_TIMEOUT
- CONTEXT_SWITCH

Request Structure:

- Issue, completion, and assignment timestamps
- Service time tracking (total + completed)
- Flags: dropped, timed_out

Event Structure:

- Time, type, request id
- Core id (for scheduling)

Global State:

- `priority_queue` `event_heap`
- vector of requests
- System metrics and counters

Config Parameters:

- Number of users, cores, threads
- Queue capacity
- Service time, think time, timeout
- Time slice for round-robin

Event Handling Logic

1. Request Arrival

- Generate new request
- Assign to thread if available
- Else enqueue or drop

2. Request Departure

- Mark completion
- Update metrics
- Schedule next request for user

3. Timeout

- Mark request as failed
- Remove from system if pending

4. Context Switch

- Preempt running request
- Reinsert into queue if unfinished

Scheduling: Round Robin

- Each request gets fixed time slice
- If not completed, preempted and requeued
- Context switch overhead included
- Ensures fairness across requests

Key Metrics:

- Average response time
- Throughput / Goodput / Badput
- CPU utilization
- Drop rate
- Average queue length
- Average number in system

Highlights from Code

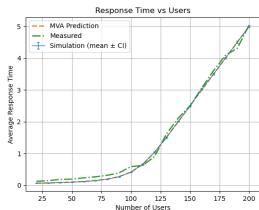
- Priority queue ensures correct event ordering
- Warm-up phase avoids bias in metrics
- Separation of goodput vs badput
- Extended for 2 server tandem queues as well

Experiment 1

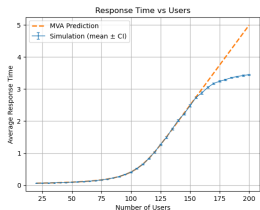
Configuration

- **System Model:** Single-server closed queueing system
- **Think Time:** Gaussian distribution ($\mu = 5s$, $\sigma = 0.5s$)
- **Service Time:** Exponential distribution (mean = 0.05s)
- **Scheduling Policy:** First-Come, First-Served (FCFS)
- **Number of Cores:** 1
- **Number of Threads:** 1

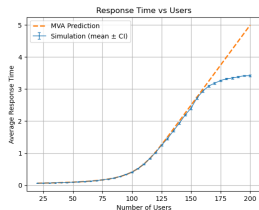
Average Response Time vs Number of Users



R vs M (infinite queue, no timeout)



R vs M (queue = 75, timeout = 4.5s)

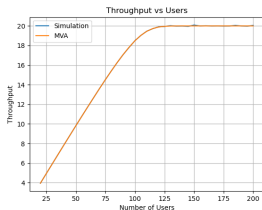
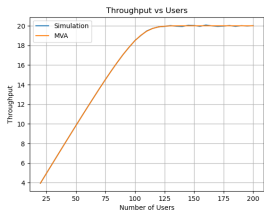
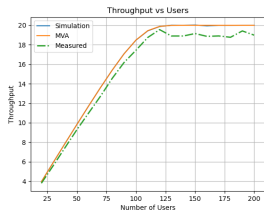


R vs M (queue = 75, timeout = 2s)

Average Response Time vs Number of Users

- For a single-core, single-threaded server, the simulation closely matches the behavior predicted by Mean Value Analysis (MVA).
- Experimental results were obtained using the k6 load generator with a PHP built-in server (single worker thread), pinned to a single core using taskset.
- Measured response times show a slight initial deviation, being marginally higher than both simulation and MVA predictions.
- With a finite queue size, response time saturates due to request drops once the queue is full, unlike MVA which assumes an infinite queue and no timeout.
- The average response times are calculated with a 99% confidence interval

Throughput vs Number of Users



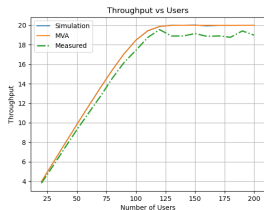
Λ vs M (infinite queue, no timeout)

Λ vs M (queue = 75, timeout = 4.5)

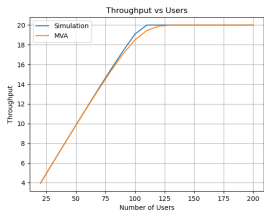
Λ vs M (queue = 75, timeout = 2)

- Simulated throughput closely matches the values predicted by Mean Value Analysis (MVA) for infinite queues.
- For finite buffer sizes, slight deviations in throughput are observed at higher user loads.
- Experimental throughput is consistently slightly lower than both simulated and MVA results.

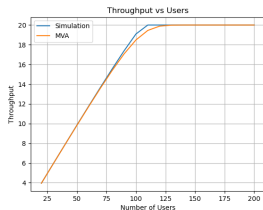
Throughput vs Number of Users



Λ vs M (Exponential Service Time, mean = 0.05)



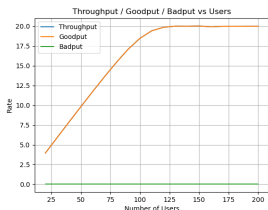
Λ vs M (Gaussian Service Time, mean = 0.05)



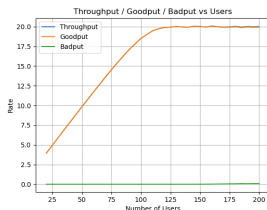
Λ vs M (Constant Service time = 0.05)

- The values obtained through mean value analysis matches with the simulated values for exponential service time.
- In gaussian and constant service time, the simulated values match the MVA values at low high load but there is a slight deviation in the middle

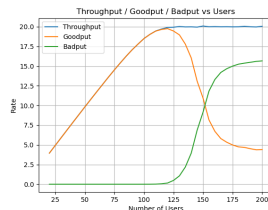
Goodput, Badput vs Number of Users



Λ_g, Λ_b vs M (infinite queue, no timeout)



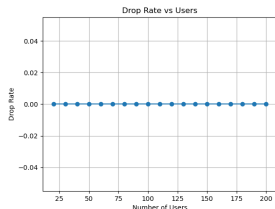
Λ_g, Λ_b vs M (queue = 75, timeout = 4.5)



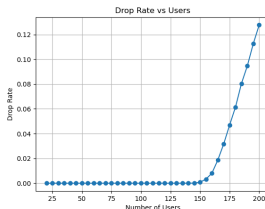
Λ_g, Λ_b vs M (queue = 75, timeout = 2)

- Badput is zero for an infinite queue with no timeout, as no requests are dropped or timed out.
- For a finite queue (length = 75) with a large timeout (4.5), badput is non-zero but remains very small, since the timeout is comparable to the maximum waiting time.
- With a smaller timeout (2), badput increases steadily while goodput decreases, as more requests time out due to longer queueing delays.

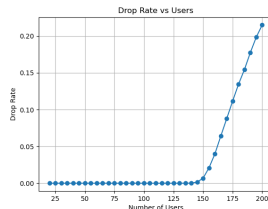
Drop Rate vs Number of Users



Drop Rate vs M (infinite queue, no timeout)



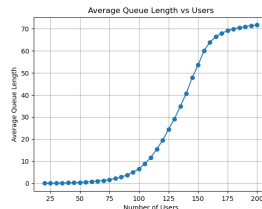
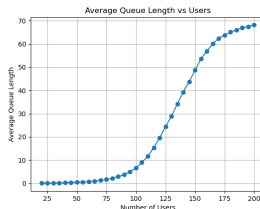
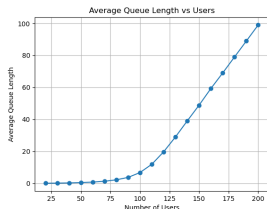
Drop Rate vs M (queue = 75, timeout = 4.5)



Drop Rate vs M (queue = 75, timeout = 2)

- For an infinite queue with no timeouts, no requests are dropped.
- For a finite queue with timeouts, requests begin to get dropped. With lower timeout values, the drop rate increases at lower user loads due to higher retry rates from timed-out requests.

Average queue length vs Number of Users



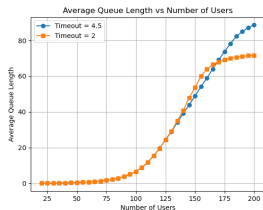
Q vs M (infinite queue, no timeout)

Q vs M (queue = 75, timeout = 4.5)

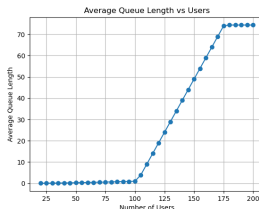
Q vs M (queue = 75, timeout = 2)

- In all cases, queue buildup begins at around $M^* \approx 100$ users.
- For finite queue length with timeouts, the average queue length saturates at 75, which is the maximum capacity.
- The point at which the average queue length saturates coincides with the point where the drop rate begins to increase.

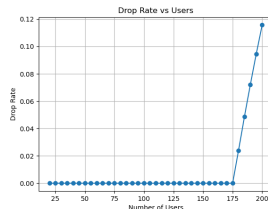
Average queue length vs Number of Users



Q vs M queue = 75



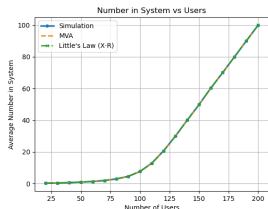
Q vs M (All Constant distribution)



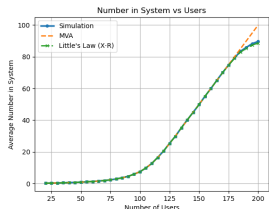
Drop Rate vs M (All constant Distribution)

- In the first graph, the average queue length increases more rapidly and saturates earlier for smaller timeout values, due to a higher rate of retries.
- In the second and third graphs, all distributions are deterministic (no randomness). Here, the queue length saturates and the drop rate begins to increase at around 175 users.
- queue saturation (and increase in drop rate) occurs when $M = M^* +$ (maximum queue length), which is 175 in this case.

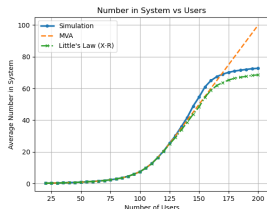
Number in System vs Number of Users



N vs M (infinite queue, no timeout)



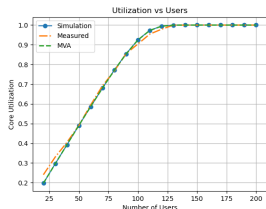
N vs M (queue = 75, timeout = 4.5)



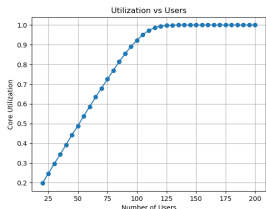
N vs M (queue = 75, timeout = 2)

- The number of requests in the system increases sharply around $M^* \approx 100$, as the queue begins to build up.
- In the first graph, the simulated values closely match those predicted by Mean Value Analysis (MVA) and Little's Law.
- In the second graph, minor deviations from MVA and Little's Law are observed, which become more pronounced in the third graph due to smaller timeout values.

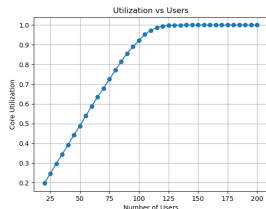
Utilization vs Number of Users



ρ vs M (infinite queue, no timeout)



ρ vs M (queue = 75, timeout = 4.5)



ρ vs M (queue = 75, timeout = 2)

- The simulated utilization match closely with that of calculated from Mean Value Analysis in the first graph with minor deviations in the experimental measured values
- The second and third graph are as expected i.e, utilization reaches 1 at $M^* = 1 + (\text{thinktime}/\text{servicetime}) \approx 100$

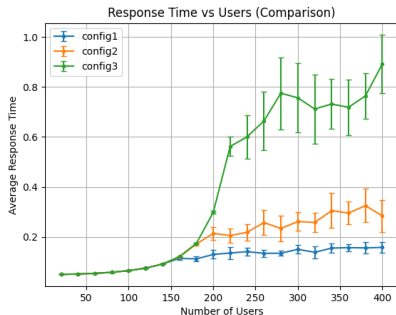
Experiment 2

Question: What happens to response time and throughput on varying threads keeping everything else same in round-robin scheduler

Configuration

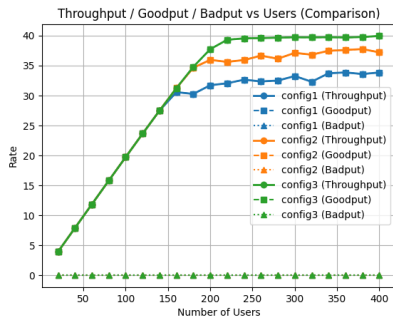
- **System Model:** Single-server closed queueing system
- **Think Time:** Gaussian distribution ($\mu = 5s$, $\sigma = 0.5s$)
- **Service Time:** Exponential distribution (mean = 0.05s)
- **Number of Cores:** 2
- **Number of Threads:** 8 vs 32 vs 64
- **Queue Capacity:** 75
- **Time slice:** 0.005s
- **Context switch Overhead:** 0.0025s

Response Time vs Number of users



- Config 1,2,3 are respectively 8, 32, 64 threads
- Response Time increases as number of threads increases because a thread has to wait for more time in the round-robin scheduler

Throughput vs Number of users



- Config 1,2,3 are respectively 8, 32, 64 threads
- Throughput increases with number of threads and reaches saturation throughput

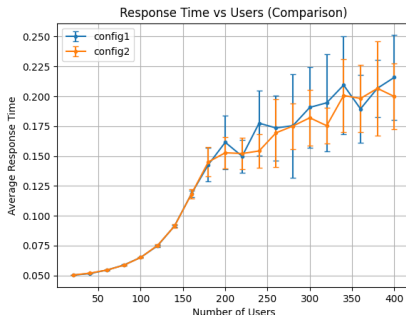
Experiment 3

Question: Average response time in FCFS vs Round-Robin

Configuration

- **System Model:** Single-server closed queueing system
- **Think Time:** Gaussian distribution ($\mu = 5s$, $\sigma = 0.5s$)
- **Service Time:** Exponential distribution (mean = 0.05s)
- **Number of Cores:** 2
- **Number of Threads:** 64
- **Queue Capacity:** 30
- **Time slice:** 10000000s(big value to simulate FCFS) vs 0.005s
- **Context switch Overhead:** 0.004s

Response Time vs Number of users



- Config 1 & 2 are respectively FCFS and Round-Robin Scheduling
- Response Time is more in FCFS because having more threads improves the scheduling via Round Robin and this effect is stronger than Context Switch Overhead

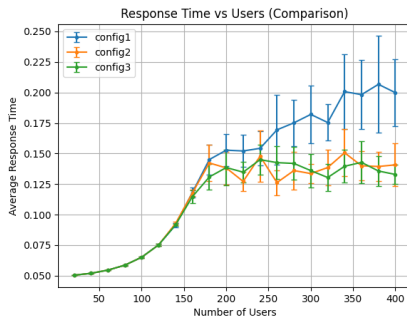
Experiment 4

Question: What happens to response time and throughput on varying Service Time Distributions(with same mean)

Configuration

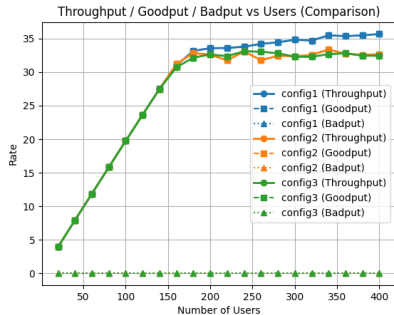
- **System Model:** Single-server closed queueing system
- **Think Time:** Gaussian distribution ($\mu = 5s$, $\sigma = 0.5s$)
- **Service Time:** Exponential (mean = 0.05s) vs Gaussian (mean = 0.05s, stddev = 0.005s) vs Constant(0.05s)
- **Number of Cores:** 2
- **Number of Threads:** 64
- **Queue Capacity:** 30
- **Time slice:** 0.005s
- **Context switch Overhead:** 0.004s

Response Time vs Number of users



- Config 1, 2, 3 are respectively Exponential, Gaussian, Constant
- Response time for Gaussian and Constant are almost same, but Response Time is significantly higher for Exponential, this could be because of high variance in Exponential

Throughput vs Number of users



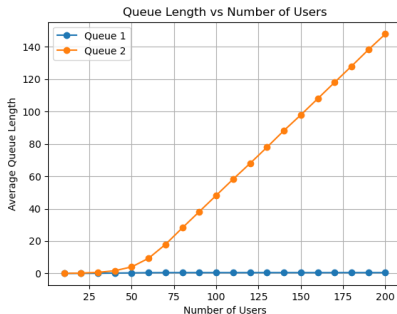
- Config 1, 2, 3 are respectively Exponential, Gaussian, Constant

Experiment 5

Configurations

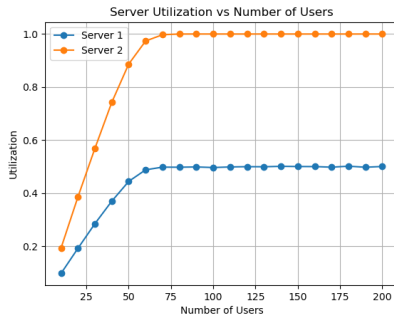
- **System Model:** Two-server closed tandem queueing system
- **Think Time:** Gaussian distribution ($\mu = 5s$, $\sigma = 0.5s$)
- **Service Time:** Exponential distribution, server1 mean = 0.05, server2 mean = 0.1
- **Number of Cores:** Server1 : 1 , Server2 : 1
- **Number of Threads:** Server : 1 , Server : 1
- All requests go to server 2 after completing their service in server 1
- Queue length is infinite and no timeouts

Avg Queue Length vs Number of users



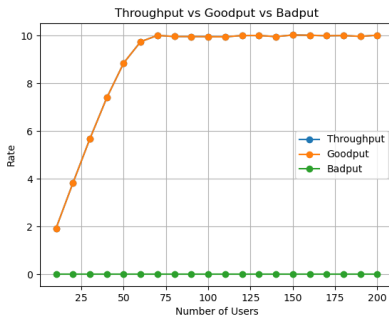
- Server 2 has more mean service time than server 1, so server 2 becomes the bottleneck and its average queue length is more than that of server 1

Avg Server Utilization vs Number of users



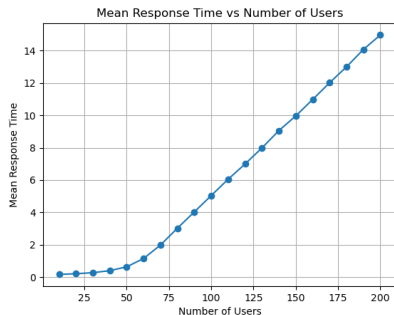
- Similar to average queue length graph, because server 2 is the bottleneck, its utilization reaches peak while server 1 operates at lower utilization

Throughput vs Number of users



- The maximum throughput is determined by the bottleneck server (Server 2). Hence, the system's maximum throughput is equal to the service rate of Server 2, i.e., $1 / (\text{mean service time of Server 2})$, which is 10.

Average Response Time vs Number of users



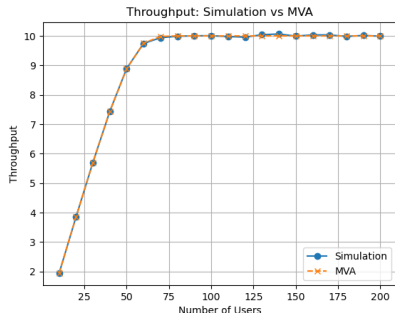
- Response time increases with number of users which is expected as the requests get queued up

Experiment 6

Configurations

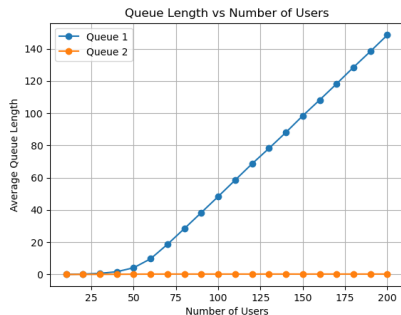
- **System Model:** Two-server closed tandem queueing system
- **Think Time:** Gaussian distribution ($\mu = 5s$, $\sigma = 0.5s$)
- **Service Time:** Exponential distribution, server1 mean = 0.1, server2 mean = 0.05
- **Number of Cores:** Server1 : 1 , Server2 : 1
- **Number of Threads:** Server : 1 , Server : 1
- All requests go to server 2 after completing their service in server 1 with a probability of 0.75 and it goes back to the user with a probability of 0.25
- Queue length is infinite and no timeouts

Throughput vs Number in System



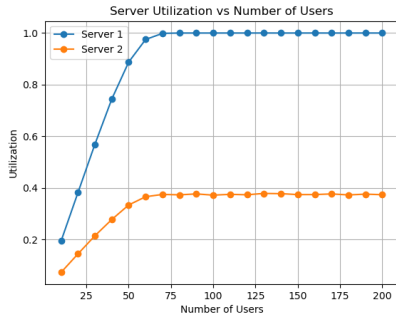
- The throughput increases as expected. Server 1 is the bottleneck so the systems maximum throughput is bounded by the maximum throughput put of server 1 which is 10
- The Simulated Throughput Matches the values obtained through Mean Value Analysis

Queue Length vs Number of Users



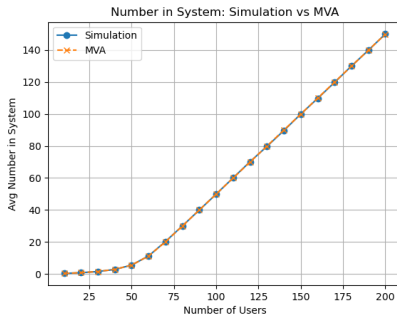
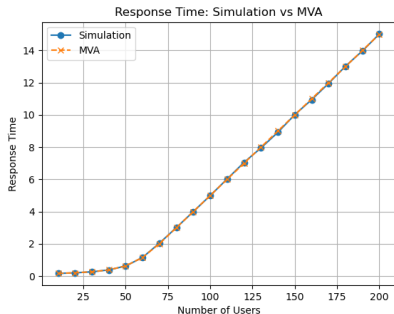
- As the server 1 is the bottleneck, its queue gradually builds up while the queue of server 2 does not build up

Server Utilization vs Number of Users



- As server 1 is the bottleneck, its utilization becomes 1 whereas server 2 will operate with a lesser utilization. The simulated utilization values match with those obtained from mean value analysis

Response Time and Number in System vs Number of users



- As expected, both will increase with number of users. The simulated values match the values obtained from mean value analysis